BARTIN ÜNİVERSİTESİ AKILLI SİSTEMLER MÜHENDİSLİĞİ ANABİLİM DALI

İleri Veritabanı Yönetim Sistemleri

İlişkisel Veritabanından NoSQL (MongoDB) Veritabanına Dinamik Veri Migrasyonu

Akdenur YEGİN - 25619824028

2025-2026 BAHAR DÖNEMİ

Veritabanı Keşif Süreci (Database Discovery)

Bu projede, kaynak veritabanına ait tablo isimleri, kolon yapıları ve kısıtlamalar (constraints) önceden bilinmeksizin sisteme dahil edilmiştir. **SQLAlchemy** kütüphanesinin inspect aracı kullanılarak çalışma anında (runtime) dinamik bir tarama yapılmıştır.

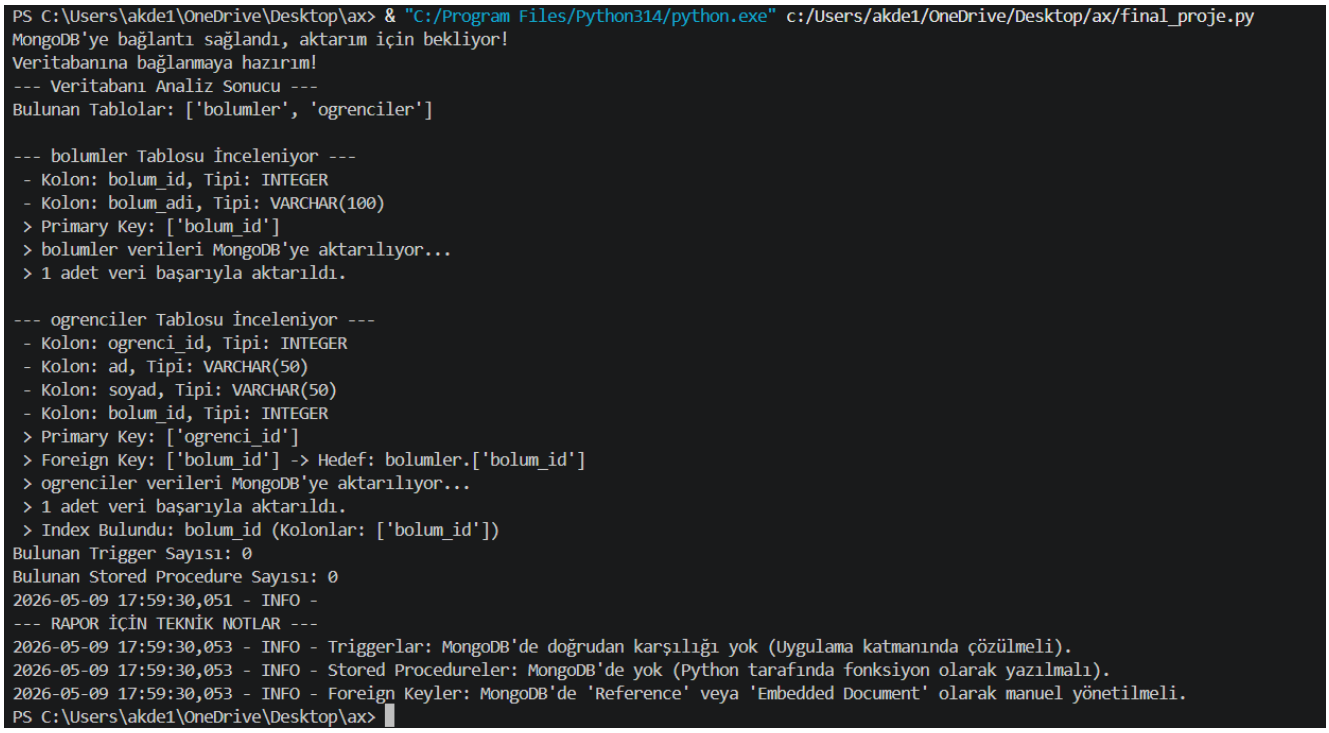
Süreç Adımları:

- Belirlenen Connection String üzerinden MySQL sunucusuna bağlantı sağlanmıştır.
- **get_table_names()** fonksiyonu ile mevcut tüm tablolar listelenmiştir.
- Her bir tablo için **get_columns()**, **get_pk_constraint()** ve **get_foreign_keys()** metodları çağrılarak ilişkisel şema haritası çıkarılmıştır.

Tespit Edilen Nesnelerin Listesi

Yapılan dinamik keşif sonucunda veritabanında bulunan tablolar ve özellikleri aşağıdaki tabloda özetlenmiştir ve şekil 1' de veritabanı nesnelerinin otomatik tespiti ve başarılı migrasyon işlemini doğrulayan sistem çıktıları gösterilmiştir:

Tablo Adı	Kolonlar	Anahtarlar (Keys)	İndeksler
bolumler	bolum_id, bolum_adi	PK: bolum_id	-
ogrenciler	ogrenci_id, ad, soyad, bolum_id	PK: ogrenci_id, FK: bolum_id	bolum_id (Index)



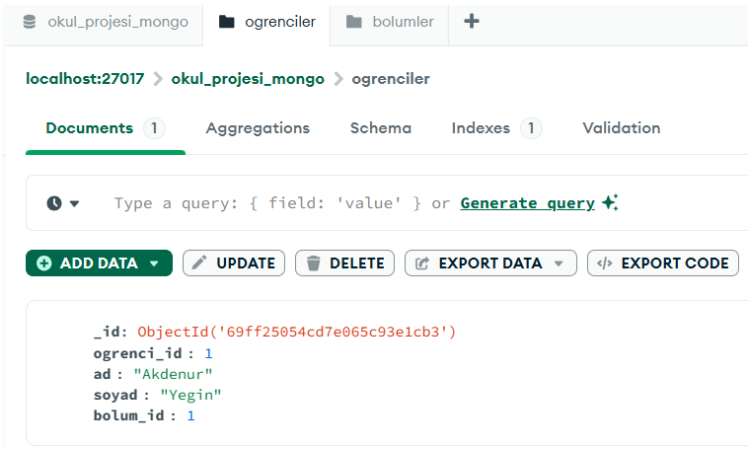
Şekil 1: Sistemin terminal çıktısı

MongoDB Veri Modeli

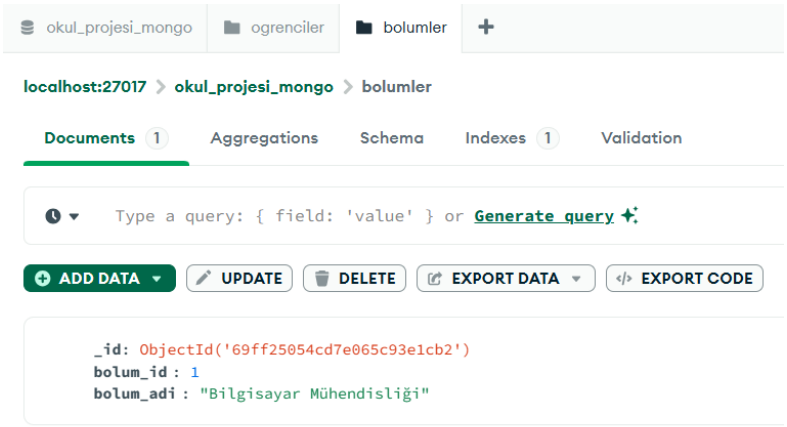
Aktarım sürecinde MySQL'deki satır-sütun tabanlı yapı, MongoDB'nin esnek doküman (document-based) modeline dönüştürülmüştür.

- **Mapping**: Her MySQL tablosu, MongoDB'de bir Collection olarak temsil edilmiştir.
- **Format:** Veriler BSON (Binary JSON) formatında saklanmış olup, ilişkiler "Flat Document" yapısıyla korunmuştur.
- **Şema Esnekliği:** MongoDB'nin şemasız (schema-less) yapısı sayesinde, ilerleyen süreçte yeni alanların eklenmesine olanak tanınmıştır.

MySQL veritabanındaki ilişkisel tablolar, MongoDB tarafında doküman tabanlı esnek bir yapıya dönüştürülmüştür. Aktarım sonrası verilerin MongoDB Compass üzerindeki doküman yapısı şekil 2 ve şekil 3' de gösterilmiştir.



Şekil 2: Ögrenciler koleksiyonu MongoDB üzerindeki JSON doküman yapısı.



Şekil 3: Bölumler koleksiyonu MongoDB üzerindeki JSON doküman yapısı.

Dönüştürülen Yapılar

Sistem, ilişkisel modelden NoSQL modeline geçiş sürecinde aşağıdaki yapıları başarıyla dönüştürmüş ve veri tutarlılığını korumuştur:

- **Veri Kayıtları (Rows to Documents):** MySQL tarafındaki ogrenciler ve bolumler tablolarındaki tüm satırlar, hiçbir veri kaybı yaşanmadan MongoDB koleksiyonları içerisine JSON dokümanları olarak aktarılmıştır.

- **Tablo Yapıları (Tables to Collections):** Kaynak veritabanındaki her bir tablo, hedef veritabanında bağımsız birer koleksiyon (Collection) olarak yeniden yapılandırılmıştır.

- **Veri Tipleri:** SQLAlchemy üzerinden okunan veriler (Integer, String vb.), MongoDB'nin desteklediği BSON tip karşılıklarına otomatik olarak eşlenmiştir.

- **İlişkisel Referanslar:** Tablolar arasındaki bolum_id üzerinden kurulan ilişkiler, MongoDB tarafında manuel referans yönetimine uygun şekilde korunmuştur.

Dönüştürülemeyen Yapılar ve Teknik Gerekçeleri

Veritabanı motorları arasındaki mimari farklar nedeniyle bazı yapılar doğrudan dönüştürülememiş, bunun yerine teknik çözüm önerileri sunulmuştur:

- **Trigger ve Stored Procedure:** MongoDB veritabanı motoru içinde SQL tabanlı tetikleyici ve prosedürleri çalıştırmaz. Gerekçe: MongoDB'nin dağıtık yapısı ve doküman odaklı felsefesi sunucu taraflı SQL mantığıyla uyumlu değildir. Öneri: Bu işlemler Python (uygulama) katmanında veya MongoDB Change Streams ile yönetilmelidir.

- **Foreign Key Constraints:** MySQL'deki katı referans bütünlüğü (Referential Integrity) MongoDB'de varsayılan olarak bulunmaz. Öneri: Veri bütünlüğü uygulama seviyesinde sağlanmalı veya ilişkili veriler "Embedded" (iç içe) doküman olarak tasarlanmalıdır.

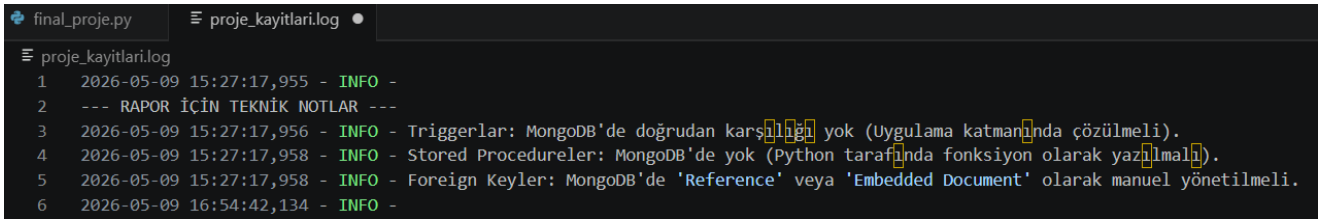
Sistem Loglama ve Hata Takip Mekanizması

Uygulamanın çalışma anındaki tüm adımları, hataları ve teknik analizleri kalıcı hale getirmek için Python logging kütüphanesi kullanılmıştır. Bu sayede:

- Uygulama kapansa dahi geçmişe dönük işlem takibi yapılabilmektedir.

- Dönüştürülemeyen yapılar (Trigger, Procedure) sistem tarafından otomatik olarak analiz edilip günlük dosyasına (proje_kayitlari.log) kaydedilmektedir.

Uygulama tarafından üretilen teknik kayıtların (log) dosya görünümü şekil 4'de gösterilmiştir.



Şekil 4: Uygulama tarafından üretilen teknik kayıtların (log) dosya görünümü.

Karşılaşılan Problemler ve Çözüm Önerileri

- **Problem:** SQLAlchemy 2.0+ sürümlerinde engine.execute() metodunun ham string sorgularını kabul etmemesi sonucu "ObjectNotExecutableError" hatası alınmıştır.

- **Çözüm:** Sorgular sqlalchemy.text fonksiyonu ile sarmalanarak çalıştırılabilir hale getirilmiş ve uyumluluk hatası giderilmiştir.

- **Mükerrer Veri Sorunu:** Uygulama her çalıştığında verilerin üst üste binmemesi için delete_many({}) fonksiyonu kullanılarak Idempotent (tekrar çalıştırılabilir) bir yapı kurulmuştur.

PROJE ERİŞİM BİLGİLERİ

Aşağıdaki bağlantılar aracılığıyla projenin kaynak kodlarına ve tanıtım videosuna ulaşılabilir:

GitHub Kaynak Kodları: https://github.com/akdenur/Dinamik_Veri_Migrasyonu

Proje Tanıtım Videosu:https://drive.google.com/file/d/1yRdIUnLXy76AeHk7aa-V7FY8T7RdgBUx/view?usp=sharing